

Algorithmes et Pensée Computationnelle

Programmation de base

Le but de cette séance est d'aborder des notions de base en programmation. Au terme de cette séance, vous serez capable de :

- Comprendre les différentes représentations de nombres.
- Définir des variables et comprendre les types de variables existants.
- Utiliser des notions d'algèbre booléenne.
- Écrire des branchements conditionnels.

Les langages qui seront utilisés pour cette séance sont Java et Python. Assurez-vous d'avoir bien installé IntelliJ. Si vous rencontrez des difficultés, n'hésitez pas à vous référer au guide suivant : [tutoriel d'installation des outils \(macOS\)](#) et [prise en main de l'environnement de travail](#) ou [tutoriel d'installation des outils \(Windows\)](#).

1 Input / Output

Question 1: (🕒 5 minutes) Output (Java et Python)

Créez une variable *nom* de type chaîne de caractères contenant votre nom, et une autre *prenom* contenant votre prénom puis affichez : "Bonjour, *prenom nom*".

💡 Conseil

Utilisez la fonction `print()` de Python et `System.out.println()` de Java.

>_ Solution

Python :

```
1 prenom = "John"
2 nom = "Doe"
3 print("Bonjour, " + prenom + " " + nom)
```

Java :

```
1 public class Question1 {
2     public static void main(String[] args) {
3         String prenom = "John";
4         String nom = "Doe";
5         System.out.println("Bonjour, " + prenom + " " + nom);
6     }
7 }
```

Question 2: (🕒 5 minutes) Input (Java et Python)

En vous référant à l'exercice précédent (**Output (Java et Python)**), demandez à l'utilisateur d'entrer son nom et son prénom au lieu d'initialiser vous-même les variables. En Java, utilisez la classe `Scanner` et la méthode `nextLine()`, et en Python, utilisez la fonction `input()`.

Conseil

En Python, utilisez la fonction `input()`, qui prend en paramètre une chaîne de caractères affichée dans la console.

En Java, utilisez la classe `Scanner` en lui passant `System.in` en paramètre, et n'oubliez pas d'ajouter `import java.util.Scanner;` tout au début de votre code.

>_ Solution

Python :

```
1 prenom = input("Quel est votre prénom ?")
2 nom = input("Quel est votre nom ?")
3 print("Bonjour, " + prenom + " " + nom)
```

Java :

```
1 import java.util.Scanner;
2
3 public class Question2 {
4     public static void main(String[] args) {
5         Scanner myScanner = new Scanner(System.in);
6
7         System.out.println("Entrez votre Prénom : ");
8         String prenom = myScanner.nextLine();
9
10        System.out.println("Entrez votre Nom : ");
11        String nom = myScanner.nextLine();
12        System.out.println("Bonjour, " + prenom + " " + nom);
13    }
14 }
```

2 Typage

Le but de cette partie des travaux pratiques est de comprendre les notions de typage statique et dynamique, ainsi que leur impact sur l'exécution et la gestion des erreurs.

Question 3: (🕒 5 minutes) Les types de variables

Quelles sont les principaux types de variables (donnez-en trois), comment les déclareriez-vous en Python et en Java ?

Conseil

Se référer aux diapositives du cours ou à la documentation officielle des langages de programmation.

>_ Solution

Les principaux types de variables sont les suivants, avec leur nom en Java indiqué entre parenthèses : `int` (`int`), `float` (`float`), `str` (`String`), `bool` (`boolean`).

En Python :

```
1 i = 0
2 f = 3.14
3 s = "Hello World"
4 b = True
```

En Java :

```
1 public class Question3 {
2     public static void main(String[] args) {
3         int i = 0;
4         float f = 3.14f;
5         String s = "Hello World";
6         boolean b = true;
7     }
8 }
```

Il existe d'autres types de variable, comme par exemple les `double`.

Question 4: (🕒 10 minutes) Typage statique et dynamique

Parmi ces différents programmes, lesquels pourront être exécutés sans problème et lesquels lèveront des erreurs ? Dans le cas où ils génèreraient des erreurs, expliquez la raison de ces erreurs.

Java :

```
1 public class Question4 {
2     public static void main(String[] args) {
3         // Les programmes ci-dessous doivent être exécutés séparément.
4         // Programme 1
5         int variableOne = 0;
6         int variableTwo = 3;
7         variableTwo = variableOne;
8
9         // Programme 2
10        var variableOne = 0;
11        var variableTwo = "Hello";
12        variableTwo = variableOne;
13
14        // Programme 3
15        int variableOne = 0;
16        String variableTwo = "Hello";
17        variableTwo = variableOne;
18
19        // Programme 4
20        var variableOne = 0;
21        variableOne = 3.14;
22
23        // Programme 5
24        var variableOne = 0;
25        String variableTwo = "Hello";
26        String variableThree = "World";
27        variableTwo = variableThree;
28    }
```

```

29         // Programme 6
30         int variableOne = 0;
31         variableOne = 3.14;
32     }
33 }

```

Python :

```

1 # Programme 7
2 variable_1 = 0
3 variable_2 = "Hello"
4 variable_2 = variable_1
5
6 # Programme 8
7 variable_1 = 0
8 variable_1 = 3.14

```

Conseil

En Java, le typage est statique : lorsqu'une variable est déclarée, un type lui est associé. Ce type peut être explicitement indiqué par le programmeur ou, dans certains cas, déduit automatiquement par Java lors de la compilation.

Une fois le type de la variable déterminé, il ne peut plus être modifié. La variable ne peut donc recevoir que des valeurs compatibles avec ce type. Si vous tentez de lui attribuer une valeur d'un type incompatible, Java signalera une erreur de compilation.

En Python, le typage est dynamique, il est donc tout à fait possible de changer le type d'une variable sans déclencher d'erreur.

Erreurs fréquentes

Au cas où vous exécuteriez les programmes Java et rencontriez l'erreur suivante : `Exception in thread "main" java.lang.UnsupportedClassVersionError: ... has been compiled by a more recent version of the Java Runtime (class file version 54.0), this version of the Java Runtime only recognizes class file versions up to 52.0`, suivez les étapes ci-dessous :

1. Mettez à jour votre JDK en téléchargeant la version adéquate via le lien suivant : <https://www.oracle.com/java/technologies/downloads/>.
2. Une fois votre JDK installé, redémarrez IntelliJ
3. Dans la barre de menus, cliquez sur **⚙ > Edit...**
4. Dans la fenêtre qui apparaîtra (capture ci-dessous), sélectionnez la version du JRE la plus récente.

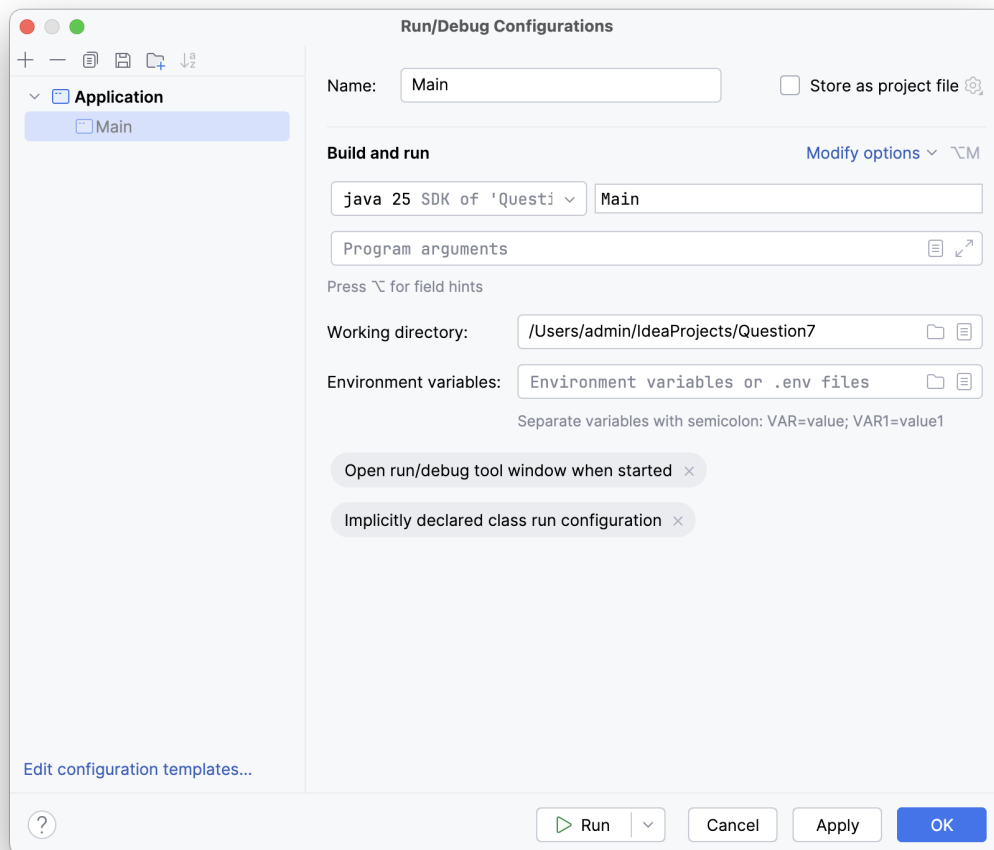


FIGURE 1 – Fenêtre de configuration de l’environnement de développement Java.

>_ Solution

En Java

- Programme 1 : OK
- Programme 2 : Ce code ne fonctionnera pas car on essaie de changer le type de la `variableTwo`, ce qui est impossible avec un typage statique comme en Java. Ici le type de la variable est déduit lors de la compilation du programme.
- Programme 3 : Tout comme le programme précédent, ce code ne fonctionnera pas car on essaye de changer le type de la `variableTwo`, ce qui est impossible avec un typage statique comme en Java.
- Programme 4 : Ce code ne fonctionnera pas car on essaie de changer le type de la `variableOne`, ce qui est impossible avec un typage statique comme en Java. Ici le type de la variable est déduit lors de la compilation du programme.
- Programme 5 : OK
- Programme 6 : Ce code ne fonctionnera pas car on essaie de changer le type de la `variableOne`, ce qui est impossible avec un typage statique comme en java.

En Python

- Programme 7 : OK
- Programme 8 : OK

3 Opérateurs et conditions booléennes

Le principe d'une valeur booléenne est qu'elle ne puisse contenir que 2 valeurs possibles, soit `True` (`true` en Java), soit `False` (`false` en Java). Il est possible de les définir en leur associant une de ces valeurs d'emblée ou de les obtenir en effectuant une opération booléenne comme par exemple une comparaison. Pour ce faire, il faut utiliser des opérateurs booléens. Voici les plus utilisés : est égal (`==` en Python et en Java), n'est pas égal (`!=` en Python et en Java), est strictement plus petit (`<` en Python et en Java), est plus petit ou égal (`<=` en Python et en Java), est strictement plus grand (`>` en Python et en Java), est plus grand ou égal (`>=` en Python et en Java). Si la condition est satisfaite, on obtiendra `True`, si elle ne l'est pas, on obtiendra `False`. L'utilisation de l'opérateur de négation (`not` en Python et `!` en Java) inversera le résultat.

Question 5: (🕒 7 minutes)

- Démontrez l'équivalence suivante :

$$(\neg p \vee q) \wedge (p \vee \neg q) \Leftrightarrow (p \wedge q) \vee (\neg p \wedge \neg q)$$

💡 Conseil

Utilisez la table des règles présentées à la diapositive 24 du cours pour déduire cette équivalence.

- Re-faire cet exercice en utilisant les tables de vérité.

>_ Solution

$$\begin{aligned}
 &(\neg p \vee q) \wedge (p \vee \neg q) && \Leftrightarrow \\
 &((\neg p \vee q) \wedge p) \vee ((\neg p \vee q) \wedge \neg q) && \Leftrightarrow \quad \text{(Distribution)} \\
 &(p \wedge (\neg p \vee q)) \vee (\neg q \wedge (\neg p \vee q)) && \Leftrightarrow \quad \text{(Commutativity)} \\
 &((p \wedge \neg p) \vee (p \wedge q)) \vee ((\neg q \wedge \neg p) \vee (\neg q \wedge q)) && \Leftrightarrow \quad \text{(Distribution)} \\
 &(F \vee (p \wedge q)) \vee ((\neg q \wedge \neg p) \vee F) && \Leftrightarrow \quad \text{(Negation)} \\
 &(p \wedge q) \vee (\neg q \wedge \neg p) && \Leftrightarrow \quad \text{(Bound)} \\
 &(p \wedge q) \vee (\neg p \wedge \neg q) && \text{(Commutativity)}
 \end{aligned}$$

>_ Solution

p	q	$\neg p$	$\neg q$	$\neg p \vee q$	$p \vee \neg q$	Partie gauche	Partie droite
T	T	F	F	T	T	T	T
T	F	F	T	F	T	F	F
F	T	T	F	T	F	F	F
F	F	T	T	T	T	T	T

Dans les exercices suivants, vous devrez anticiper la valeur que la console va vous donner (résultat du print).

Question 6: (🕒 5 minutes) Qu'affiche le programme suivant ?

💡 Conseil

Utilisez les tables de vérité présentées à la diapositive 23 du cours.

- `a = 3`
- `b = 2`
- `c = 6`

```

4 d = c >= b
5 print(a==b or d)
6 print(a==b and d)
7 print(a==b or a==c or False or d)
8 print(a<c and not (4*b==8 and not d))

```

>_ Solution

```

True
False
True
True

```

Question 7: (🕒 5 minutes)

1. Qu'affiche le programme suivant ?

```

1 a = True
2 b = False
3 c = True
4 if a or (b and not c):
5     print("output 1")
6 if b!= c :
7     print("output 2")
8 if a or (not b != (c and a)):
9     print("output 3")
10 else:
11     print("output 4")

```

2. Qu'affiche le programme si l'on remplace la ligne 6 par `elif b != c:` ?

>_ Solution

```

1. output 1
   output 2
   output 3
2. output 1
   output 3

```

4 Match (Python) et Switch (Java)

L'instruction `match` en Python et `switch` en Java a pour objectif de simplifier la logique conditionnelle en sélectionnant un bloc de code à exécuter parmi plusieurs options. Cette structure compare la valeur d'une variable ou d'une expression à une série de cas (case) prédéfinis. Lorsqu'une correspondance est trouvée, le code associé à ce cas est exécuté, offrant ainsi une alternative plus lisible et organisée qu'une longue chaîne de `if-elif-else`.

Il existe toutefois une différence importante entre les deux langages : avec la syntaxe classique de `switch` en Java utilisant les deux-points, l'exécution peut continuer dans les `case` suivants tant qu'aucune instruction `break` n'est rencontrée. Ce comportement est appelé *fall-through*. En Python, `match` n'a pas ce comportement : seul le premier `case` correspondant est exécuté.

Question 8: (🕒 5 minutes) Écrire le code Java qui demande à l'utilisateur d'entrer l'étape d'exécution d'une instruction et affiche les étapes qui restent à exécuter (y compris l'étape courante). Si l'utilisateur entre une étape invalide le code affiche `L'étape n'est pas valide`.

💡 Conseil

Rappelez-vous l'architecture de von Neumann disponible à la diapositive 17 du [premier cours](#). Pour lire la saisie texte de l'utilisateur, utilisez `sc.nextLine()` après avoir écrit `Scanner sc = new Scanner(System.in)` (n'oubliez pas d'ajouter `import java.util.Scanner;` tout au début de votre code).

Exemple

Si l'utilisateur entre `execute`, le programme va afficher :

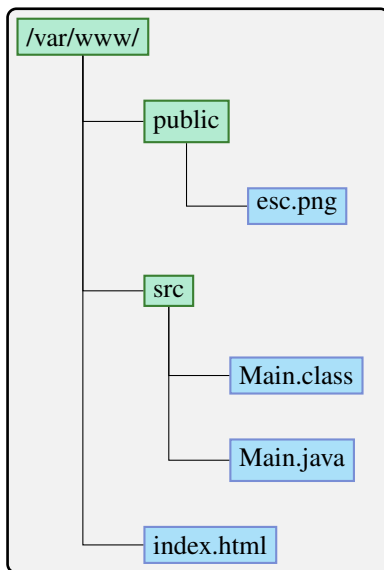
L'étape `execute` s'exécute ou va être exécutée.

L'étape `store` s'exécute ou va être exécutée.

>_ Solution

```
1  import java.util.Scanner;
2
3  public class Question8 {
4      public static void main(String[] args) {
5          Scanner scanner = new Scanner(System.in);
6          String step = scanner.nextLine();
7          switch (step) {
8              case "fetch":
9                  System.out.println("L'étape fetch s'exécute ou
10 va être exécutée.");
11              case "decode":
12                  System.out.println("L'étape decode s'exécute ou
13 va être exécutée.");
14              case "execute":
15                  System.out.println("L'étape execute s'exécute
16 ou va être exécutée.");
17              case "store":
18                  System.out.println("L'étape store s'exécute ou
19 va être exécutée.");
20                  break;
21              default:
22                  System.out.println("L'étape n'est pas valide");
23          }
24      }
25  }
```

Question 9: (🕒 5 minutes)



Soit l'arborescence à gauche d'un système de fichiers (file-system). Écrivez un programme Python (utilisant `match`) qui affiche **seulement les noms des fichiers** dans un seul dossier spécifié à la ligne de commande. Votre programme doit seulement gérer les arguments suivants :

- `/var/www/./public/..`
- `/var/www/src/..`
- `/var/www/src/`
- `/var/www/public/`

Pour les autres cas, il faut afficher La saisie n'était pas reconnue.

💡 Conseil

Pour spécifier un argument à la ligne de commande en Python [consultez cette documentation en ligne](#). Pour le `match` en Python, consultez [la diapositive 32 du cours](#).

Re-faire cet exercice en utilisant `input` pour lire la saisie de l'utilisateur.

💡 Conseil

La documentation de `input` est [disponible sur ce lien](#).

>_ Solution

```
1 import sys
2
3 match sys.argv[1]:
4     case "/var/www/./public/..":
5         print("index.html")
6     case "/var/www/src/..":
7         print("index.html")
8     case "/var/www/src/":
9         print("Main.class\nMain.java")
10    case "/var/www/public/":
11        print("esc.png")
12    case _: print("La saisie n'était pas reconnue")
```

>_ Solution

```
1 import sys
2
3 entry = input("Entrez le chemin: ")
4 match entry:
5     case "/var/www/./public/..":
6         print("index.html")
7     case "/var/www/src/..":
8         print("index.html")
9     case "/var/www/src/":
10        print("Main.class\nMain.java")
11    case "/var/www/public/":
12        print("esc.png")
13    case _: print("La saisie n'était pas reconnue")
```

5 Type casting

Question 10: (🕒 5 minutes) Conversion des variables (Type casting) (Java et Python)

Qu'affichent les programmes suivants ?

Python :

```
1 nombre_entier = 3
2 nombre_decimal = float(nombre_entier)
3 print(nombre_entier)
4 print(nombre_decimal)
```

Java :

```
1 public class Question10 {
2     public static void main(String[] args) {
3         float nombreDecimal = 3.14f;
4         int nombreEntier = (int) nombreDecimal;
5         System.out.println(nombreEntier);
6         System.out.println(nombreDecimal);
7     }
8 }
```

💡 Conseil

Attention, ces fonctions ne changent pas le type des variables, elles ne font que les convertir.

>_ Solution

Python :

3
3.0

Java :

3
3.14

6 Représentation de nombres entiers

Question 11: (🕒 5 minutes) Entiers non signés

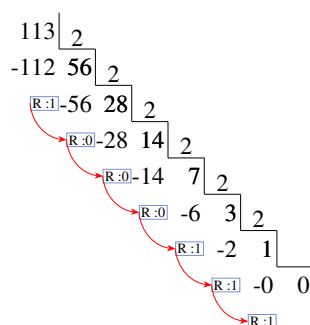
Sur 8 bits, convertir $113_{(10)}$ en base binaire.

💡 Conseil

Faire un tableau comme présenté dans la diapositive 10 du cours (programmation de base). Essayer de décomposer le nombre en une somme de puissances de 2.

>_ Solution

Méthode 1 : Utiliser la division par 2 comme vu dans la première séance d'exercices.



Méthode 2 : Utiliser les puissances de 2 pour décomposer le nombre (vu également dans la première séance d'exercices).

$$113 = 64 + 32 + 16 + 1 = 1 * 2^6 + 1 * 2^5 + 1 * 2^4 + 1 * 2^0$$

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
128	64	32	16	8	4	2	1
0	1	1	1	0	0	0	1

On obtient donc $01110001_{(2)}$

Question 12: (🕒 5 minutes) Entiers signés négatifs

En utilisant le résultat de la question précédente, convertir sur 8 bits $-113_{(10)}$ en base binaire. Utiliser la représentation **signée** à la diapositive 10 du cours (et non le complément à 1 ou 2).

💡 Conseil

- Il faut prendre la représentation sur 7 bits d'un nombre entier non signé.
- On rajoute un 8ème bit qui sera le signe.
- Le premier bit à gauche vaut 0 pour un nombre positif et 1 pour un nombre négatif.

>_ Solution

$$113_{(10)} = 01110001_{(2)}$$

En changeant le premier bit à 1 pour faire passer le nombre en nombre négatif on obtient :

$$-113_{(10)} = 11110001_{(2)}$$

Question 13: (🕒 5 minutes) Complément à 1

Écrire $-113_{(10)}$ en complément à 1 sur 8 bits.

Quelle est la différence entre cette méthode et la précédente ?

💡 Conseil

- Pour inverser les bits, remplacer chaque 0 par 1 et chaque 1 par 0.
- Pour étudier la différence, il faut regarder les différentes manières d'exprimer -0 en binaire (se référer à la diapositive 11 du cours).

>_ Solution

$$113_{(10)} = 01110001_{(2)}$$

0	1	1	1	0	0	0	1
not	not	not	not	not	not	not	not
1	0	0	0	1	1	1	0

Avec cette méthode, $-113_{(10)} = 10001110_{(2)}$

L'intervalle de cette méthode ne change pas par rapport à la précédente. Par contre, l'expression de -0 sera différente.

Signé : $-0_{(10)} = 10000000_{(2)}$

Complément à 1 : $-0_{(10)} = 11111111_{(2)}$

Les deux ont le même intervalle : $[-127_{(10)}, +127_{(10)}]$

Question 14: (🕒 5 minutes) Complément à 2

Écrire $-113_{(10)}$ en complément à 2 sur 8 bits, et expliquer l'utilité de cette représentation.

💡 Conseil

Exemple du cours avec $87_{(10)}$

$$87_{(10)} = 01010111_{(2)}$$

a	0	1	0	1	0	1	1	1
b	1	0	1	0	1	0	0	0
c	1	0	1	0	1	0	0	1

a : convertir le nombre en binaire

b : inverser tous les bits (en utilisant **not**)

c : rajouter 1 à b pour obtenir le complément à 2

On obtient donc $-87_{(10)} = 10101001_{(2)}$

>_ Solution

Ici, il suffit donc d'ajouter 1 au complément à 1 de $-113_{(10)}$

On obtient donc $10001111_{(2)}$

L'intervalle change, car en complément à 2, $+0_{(10)}$ et $-0_{(10)}$ ne disposent plus de deux représentations distinctes.

Le nouvel intervalle est donc $[-128_{(10)}, +127_{(10)}]$

Question 15: (🕒 10 minutes) Nombres à virgule flottante (Floating point)

Voici la représentation en binaire d'un nombre à virgule flottante :

signe	exposant	mantisse
0	10110101	010000010000000000000001

Que vaut cette représentation en base 10 ? Utiliser la représentation des nombres à virgule flottante (avec un biais de 127). Arrondir les résultats intermédiaires et la valeur finale au 3ème chiffre significatif après la virgule.

💡 Conseil

- Se référer à la diapositive 16 du cours pour plus de détails.
- Poser chaque calcul, et ensuite tout fusionner avec la formule.
- L'exposant et la mantisse sont des entiers positifs non signés.
- Pour vérifier votre résultat, vous pouvez utiliser l'outil suivant : <https://www.h-schmidt.net/FloatConverter/IEEE754.html>

>_ Solution

- Signe du nombre : $(-1)^{\text{signe}} = (-1)^0 = 1$
- Exposant en base 10 : $10110101_{(2)} = 181_{(10)}$
- Pour obtenir l'exposant, il faut encore lui appliquer le biais, il faut soustraire 127 à notre résultat. Ici on aura $2^{(181-127)} = 2^{54}$
- Mantisse = $1 + 010000010000000000000001_{(2)} = 1 + 1 \cdot 2^{-2} + 1 \cdot 2^{-8} + 1 \cdot 2^{-23} \approx 1.254_{(10)}$
- Valeur = Signe * Mantisse * $2^{\text{exposant}-127} \approx 1 * 1.254 * 2^{54} = 2.259 * 10^{16}$

7 Chaîne de caractères

Question 16: (🕒 10 minutes)

Soit la chaîne de caractères : UNIL\nSchool_of_Criminal_Sciences.

myString="	U	N	I	L	\n	S	c	h	o	...	"
indices	0	1	2	3	4	5	6	7	8	9 à 31	

⚠ Attention

Ici, le caractère `_` est seulement pour mettre en évidence le caractère espace.

1. Affichez cette chaîne en Python et en Java. Pourquoi est-elle séparée par une nouvelle ligne ?
2. Essayez d'afficher uniquement la chaîne `School of Crimi` en sélectionnant une sous-chaîne de la chaîne initiale.

💡 Conseil

- Pour créer une sous-chaîne en Java, utilisez la méthode `myString.substring(int beginIndex, int endIndex)` où `beginIndex` est l'indice du premier caractère dans la chaîne `myString` que l'on veut inclure dans la sous-chaîne et `endIndex` est l'indice du caractère suivant le dernier à inclure (le caractère à l'indice `endIndex` n'est pas inclus dans la sous-chaîne).
- Pour créer une sous-chaîne en Python, utilisez l'opérateur slice `myString[beginIndex:endIndex]` et pareil pour l'interprétation du `beginIndex`, `endIndex`, et `myString`.

3. Affichez le troisième caractère de la chaîne en Python et en Java.

💡 Conseil

- Pour obtenir le *i*ème caractère en Java, on écrit `myString.charAt(i)`.
- Pour obtenir le *i*ème caractère en Python, on écrit `myString[i]`.

4. Afficher la longueur de la chaîne de caractères en Python et en Java.

💡 Conseil

Utiliser la méthode `myString.length()` en Java et la fonction `len` en Python.

>_ Solution

Pour la question 1, la chaîne s'affiche sur deux lignes à cause de la séquence d'échappement `\n`, qui insère un retour à la ligne.

>_ Solution

```
1 s = "UNIL\nSchool of Criminal Sciences"
2 print(s)
3 print(s[5:20])
4 print(s[2])
5 print(len(s))
```

>_ Solution

```
1 public class Question16 {
2     public static void main(String[] args) {
3         String s = "UNIL\nSchool of Criminal Sciences";
4
5         System.out.println(s); // \n crée un saut de ligne
6
7         // Imprimer uniquement la sous-chaine School of crimi.
8         System.out.println(s.substring(5, 20));
9
10        // Accéder au troisième caractère
11        System.out.println(s.charAt(2)); // 'I'
12
13        System.out.println(s.length());
14    }
15 }
```